MIVA OPEN UNIVERSITY — SCHOOL OF POSTGRADUATE STUDIES

Master of Information Technology (MIT) | MIT 8103: Advanced Database Systems

Student Name: Peace Ekoh
Student ID: 301866431
Academic Session: 2026/2027 (First Semester CA)
Submission Platform: LMS & GitHub
GitHub Repository: https://github.com/EKOHPEACE/Peace_Ekoh_301866431

CASE STUDY: NOVAMART RETAIL STOREFRONT & LOGISTICS

NovaMart is an omnichannel e-commerce retail and logistics platform modeled after contemporary digital marketplaces operating in Nigeria (such as Jumia and Konga). The system records customers, product categories, products, orders, order items, inventory logs, and payments. The platform handles daily activities across four core departments: Consumer Electronics, Groceries & Provisions, Fashion & Apparel, and Home Appliances.

The database supports three primary user groups: shoppers browsing products, placing orders, and making digital payments; warehouse staff tracking stock levels, receiving restocks, and updating safety buffers; and financial analysts verifying payments and auditing order balances. To support these operations efficiently, NovaMart implements a **polyglot persistence architecture**: MySQL 8.0 (InnoDB) manages ACID transactional ledgers, MongoDB stores the flexible product catalog and customer reviews, and Redis provides in-memory cart sessions and real-time best-seller leaderboards.

PORTFOLIO 1: DATABASE DESIGN AND MODELLING

The relational schema consists of eight tables implemented in MySQL 8.0 on the InnoDB engine: `customers`, `customer_addresses`, `categories`, `products`, `orders`, `order_items`, `payments`, and `inventory_logs`.

- **Key & Constraint Architecture:** Integer surrogate primary keys are defined across all tables. Referential integrity is enforced with foreign keys using `ON DELETE RESTRICT` on products and categories to prevent orphaned records, and `ON DELETE CASCADE` on customer addresses. Unique constraints enforce distinct emails and SKUs. Check constraints enforce positive pricing (`unit_price > 0`) and non-negative stock (`stock_quantity >= 0`). An automated `AFTER UPDATE` trigger (`trg_audit_product_stock_change`) records every inventory movement into `inventory_logs` with before and after quantities.
- **Normalisation Walkthrough (1NF to 3NF/BCNF):**
  - **1NF:** All column attributes are atomic. Repeating order item groups and multi-valued customer addresses were eliminated by establishing dedicated junction and address tables.
  - **2NF:** In the composite relation `order_items(order_id, product_id)`, partial key dependencies were resolved by decoupling product descriptions and categories into `products`, keeping only transaction-specific quantity and purchase price in `order_items`.
  - **3NF / BCNF:** Transitive dependencies (`order_id → customer_id → customer_email` and `product_id → category_id → category_name`) were eliminated by isolating categories and customer profiles into independent tables, preventing update and deletion anomalies.
- **Operational Views:** Implemented views `vw_low_stock_alerts` (real-time restocking deficit calculation) and `vw_customer_order_metrics` (customer lifetime spend and tier classification) to abstract underlying joins for operational users.

PORTFOLIO 2: QUERY PROCESSING AND OPTIMISATION

To test query execution under realistic enterprise volumes, a stored procedure populated `novamart_db` with **9,010 Orders**, **9,015 Order Items**, and **9,010 Payments**. Analytical queries were authored using multi-table joins, subqueries, grouping, and window functions (`DENSE_RANK()`, `LAG()`) to evaluate profit margins and month-over-month growth.

Execution Plan Tuning (`EXPLAIN ANALYZE`): A frequently run portal query filtering delivered orders within a date range and sorting by date descending was benchmarked before and after indexing:

Measurement	Before Index	After Composite Index	After Covering Index
Access Type	Single-column Index Scan + Filesort	Composite Index Range Scan	Covering Index Only Scan
Rows Scanned	3,802 rows	1,280 rows (66.3% reduction)	1,280 rows
Execution Time	25.47 ms	14.74 ms (42.1% speedup)	3.07 ms (87.9% speedup)
Query Cost	915.60	480.20	128.40

The composite index `idx_orders_status_date(order_status, order_date DESC)` eliminated the expensive in-memory filesort by ordering equality columns before range/sorting columns. The covering index `idx_orders_covering` satisfied all projected attributes directly from RAM leaf nodes, avoiding underlying table reads.

PORTFOLIO 3: TRANSACTIONS AND CONCURRENCY CONTROL

- **ACID Demonstrations:**
  - **Atomic Checkout (COMMIT):** Decremented stock for an iPhone (25 → 24), inserted order `9388`, line items, and payment `PAY-ACID-SUCCESS-9388`, committing atomically.
  - **Failed Payment (ROLLBACK):** Simulated a payment gateway decline (HTTP 402) on a MacBook Pro. An immediate `ROLLBACK` restored stock to 10 units with zero orphaned order records.
  - **Partial Recovery (SAVEPOINT):** Simulated digital wallet failure; transaction rolled back to `SAVEPOINT before_payment_attempt` and completed via Credit Card without re-executing order reservation.
- **Concurrency & Row-Lock Proof (`simulate_concurrency.py`):** Two parallel threads competed for Product 8 (refrigerator, initial stock = 6) using pessimistic locking (`SELECT ... FOR UPDATE`). Session 1 locked the row at `13:42:59`. Session 2 attempted access at `13:42:59.692` and was deterministically blocked in a `LOCK WAIT` queue for **2.64 seconds** until Session 1 committed. Session 2 then safely read updated stock (5) and decremented to **4**, proving complete elimination of the lost-update race condition.
- **Lock Monitoring:** Live contention was inspected via `information_schema.innodb_trx`, `performance_schema.data_locks`, and `data_lock_waits`.

PORTFOLIO 4: NOSQL AND ADVANCED DATA MODELS

- **Document Model (MongoDB):** Designed polymorphic JSON documents in `novamart_catalog.products`. Nested `specifications` store divergent attributes (RAM/CPU for laptops, virgin wool for suits, expiration dates for groceries) without relational EAV tables. Customer reviews are embedded directly inside product documents for fast $O(1)$ page reads.
- **Atomic Updates & Aggregations:** Demonstrated atomic review insertions using `$push` and `$inc`, and executed a 4-stage aggregation pipeline calculating category satisfaction metrics (Electronics: 4.94★ across 6 reviews; Groceries: 4.90★ across 2 reviews; Fashion: 4.70★).
- **In-Memory Key-Value Caching (Redis 7.0):** Modeled ephemeral shopping carts with a 7-day TTL (`HSET cart:session:<id>` with `EXPIRE 604800`), flash-sale atomic stock decrements (`DECRBY`), and real-time best-seller leaderboards using Redis Sorted Sets (`ZSET`).

PORTFOLIO 5: DISTRIBUTED AND CLOUD DATABASE EXERCISE

- **Distributed Topology:** Configured a multi-container cluster via Docker Compose (`Peace_Ekoh_301866431_P5_DockerCluster.yml`) featuring a primary MySQL master with Global Transaction Identifiers (GTID) and binary logging, paired with a dedicated read-only replica and an in-memory Redis cache node.
- **Disaster Recovery Failover Drill:** Simulated an Availability Zone failure in `Peace_Ekoh_301866431_P5_FailoverDrill.sql`. Flushed queued relay logs, severed the replication link, lifted read-only locks (`SET GLOBAL read_only = OFF`), and executed an audit transaction within an empirical Recovery Time Objective (RTO) of **14.82 seconds** (comfortably below the 30-second target).
- **Horizontal Sharding & CAP Theorem:** Evaluated an entity-based horizontal sharding strategy partitioned by `customer_id` using consistent hashing ($\text{Shard} = \text{MurmurHash3}(\text{customer_id}) \pmod N$), ensuring 98% of customer queries remain single-shard operations. Under the CAP theorem, the financial ledger prioritizes **CP** (Consistency + Partition Tolerance), while product browsing prioritizes **AP** (Availability + Eventual Consistency).

EVIDENCE

Executable scripts, schemas, benchmark logs, and execution evidence are organized across the repository:

- **Portfolio 1 (Design & DDL):** portfolio-1-database-design
- **Portfolio 2 (Optimization & Plans):** portfolio-2-query-optimisation
- **Portfolio 3 (Transactions & Concurrency):** portfolio-3-transactions-concurrency
- **Portfolio 4 (NoSQL & Caching):** portfolio-4-nosql
- **Portfolio 5 (Distributed & Replication):** portfolio-5-distributed-cloud

INDIVIDUAL REFLECTION

Decoupling the architecture into relational MySQL for financial ledgers and MongoDB for the product catalog was the most significant design choice. Trying to handle polymorphic specifications in MySQL using JSON columns resulted in unwieldy queries and poor indexing performance. In contrast, MongoDB handles schemaless product variations naturally.

During implementation, bulk seeding initially failed with `ERROR 1292: Incorrect datetime value` because random timestamp generation in standard time zones hit non-existent hour values during Daylight Saving Time spring-forward transitions. I resolved this by enforcing explicit UTC handling (`SET time_zone = '+00:00';`) at the session level. Profiling queries with `EXPLAIN ANALYZE` demonstrated that composite indexes must order equality filters before range columns to eliminate in-memory sorting. Finally, simulating concurrent transactions highlighted that while pessimistic row locking prevents race conditions, minimizing lock hold duration is essential to avoid connection exhaustion in high-traffic environments.

REFERENCES

1. Oracle Corporation. *MySQL 8.0 Reference Manual: InnoDB Locking and Transaction Model*. Available at: <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking.html>.
2. Oracle Corporation. *MySQL 8.0 Reference Manual: Optimization and Indexes*. Available at: <https://dev.mysql.com/doc/refman/8.0/en/optimization.html>.
3. MongoDB Inc. *MongoDB Documentation: Data Modeling and Aggregation Pipeline*. Available at: <https://www.mongodb.com/docs/>.
4. Redis Ltd. *Redis Documentation: Sorted Sets and Hashes*. Available at: <https://redis.io/docs/>.
5. Kleppmann, M., 2017. *Designing Data-Intensive Applications*. Sebastopol: O'Reilly Media.

AI-USE DECLARATION

> *"I used Google DeepMind / Antigravity Agentic Assistant to explain database concepts, brainstorm query scenarios, and help review the structure of my portfolio. I executed the database commands myself, checked the results against the assessment requirements, and validated issues before submission. I understand the work and remain responsible for its accuracy."*